

西南交通大學
Southwest Jiaotong University

Operating System

Fei Teng
fteng@swjtu.edu.cn

Content

1. Processor Management
2. Process
3. **Thread**
4. Scheduling （调度）
5. Concurrency (并发)
6. Synchronization and Deadlock



3.1 Process and Thread

- Multithreading
- Thread functionality

Program Example

Sum of all the numbers of a dataset (S):

Single Thread

```
A = 0
for i in S:
    A += i
```

Multiple Threads

```
A1, A2, …, An = 0
Split S into multiple set (S1, S2, …, Sn)
for i =1 to n:
    initial and begin thread_i to obtain the sum of Si
threads join()
Sum A1, A2,…, An
```

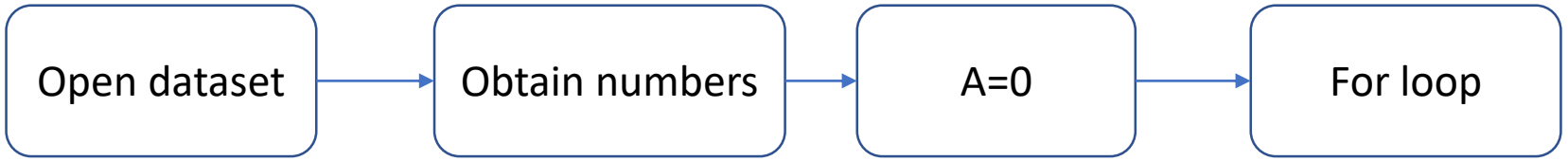
Program Example

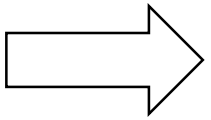
Sum of all the numbers of a dataset (S):

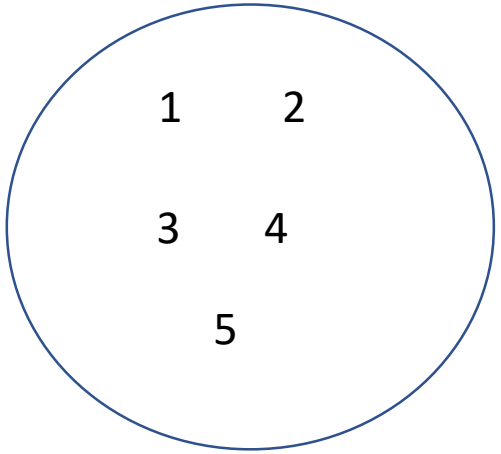
Single Thread

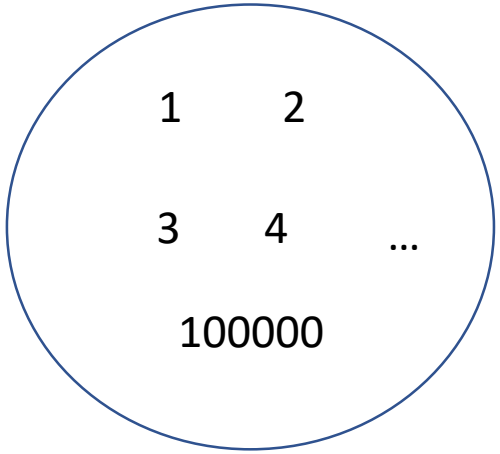
```
A = 0
for i in S:
    A += i
```

Sequential execution: Subsequent operations can only be executed after the current operation (program segment) is completed







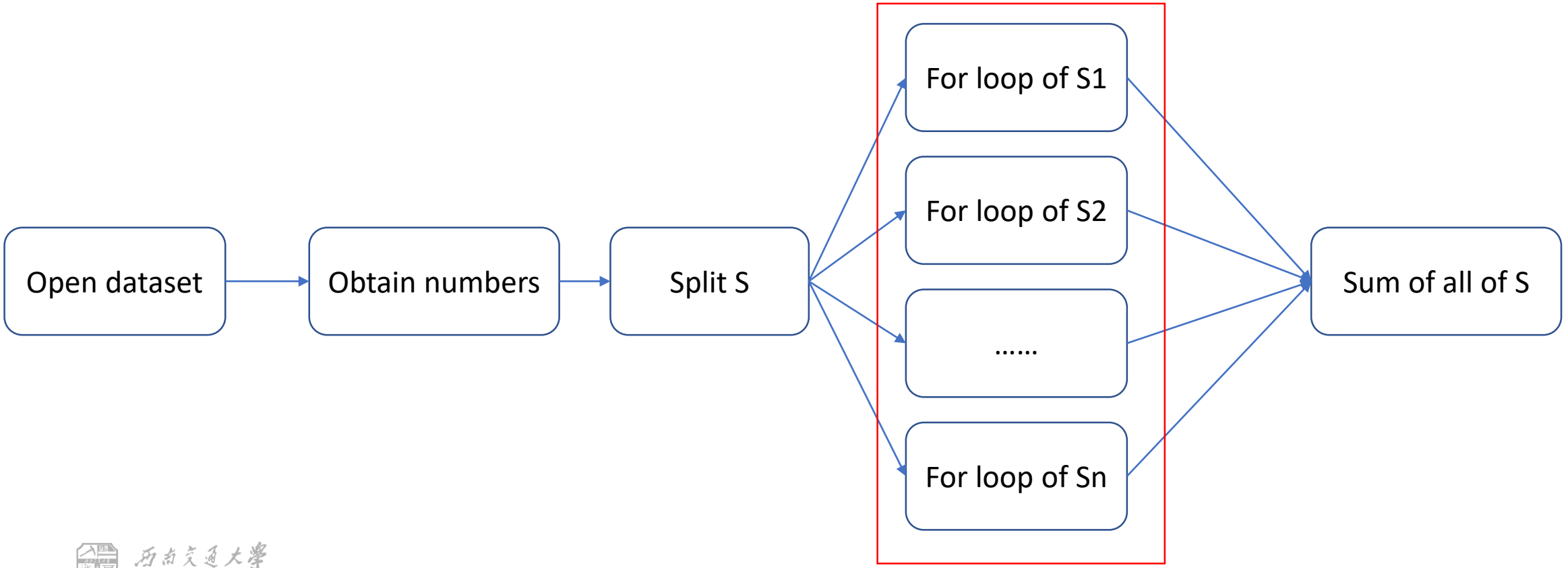


Different data can lead to different results with the same program

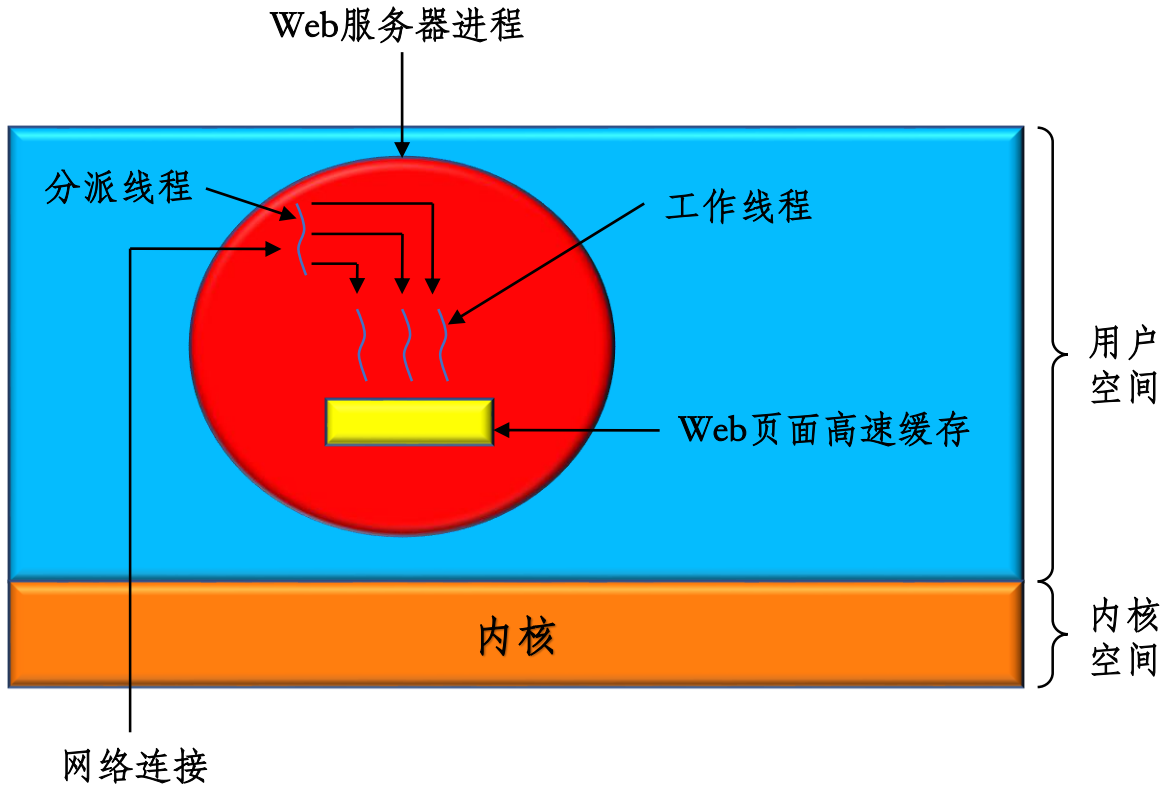
Program Example

Sum of all the numbers of a dataset (S):

Multiple Threads: multiple threads can run simultaneously (parallel or concurrent)



A multi threading wed server




```
while(TRUE){
    get_next_request(&buf);
    handoff_work(&buf);
}
```

a)分派线程

```
while(TRUE){
      wait_for_work(&buf);
      look_for_page_in_cache(&buf,&page);
      if(page_not_in_cache(&page);
        read_page_form_disk(&buf,&page);
      return_page(&page);
}
```

b)工作线程

A Mp3 application

```
main( )
{
        while(TRUE )
        {
              Read( );
              Decompress( );
              Play( );
         }
}
Read( ) { ... }
Decompress( ) { ... }
Play( ) { ... }
```

A Mp3 application



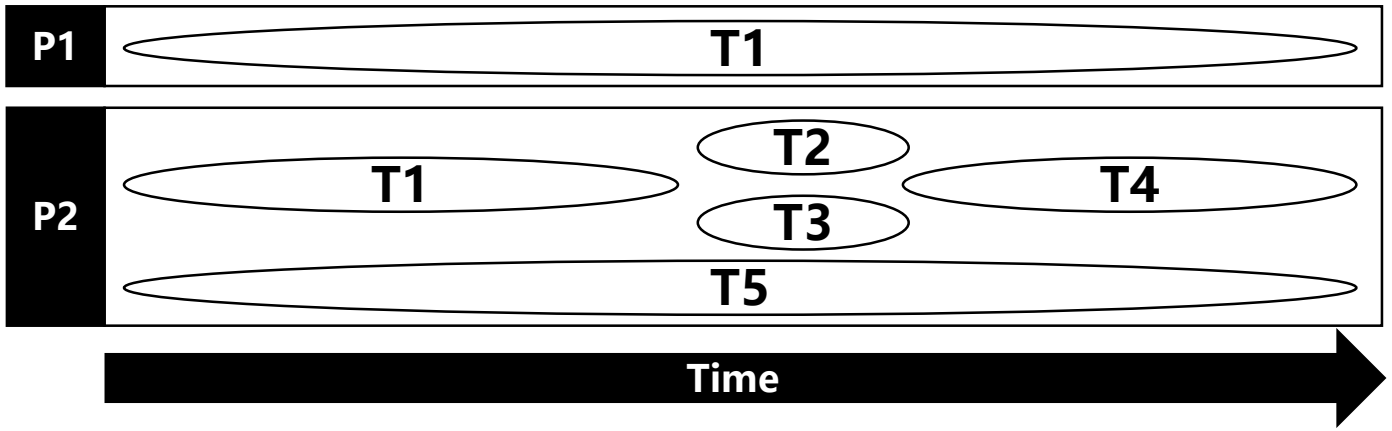
Why MultiThreading?

- **The ability of parallel entities to share an address space and all available data**
- **Threads are lighter than processes, making them easier (i.e. faster) to create and easier to undo**
- **Due to performance reasons, such as the ability of multiple threads to overlap CPU and I/O**
- **In multiple CPUs, multithreading can achieve parallelism**

Process and Thread

- **Process 进程**
  - **The definition of a process: A process is one-time execution process of a concurrently executable program with independent functionality regarding a certain dataset, also the basic unit for resource allocation and protection in operating systems. (1978 National Conference on Operating Systems)**

- **Thread 线程**
  - **The definition of a thread: A thread is an execution unit (a single sequential control flow) in a process, responsible for the execution of programs in the current process. There is at least one thread in a process.**



Process Properties

- **Definition of a process: A process is one-time execution process of a concurrently executable program with independent functionality regarding a certain dataset, also the basic unit for resource allocation and protection in operating systems. (1978 National Conference on Operating Systems)**

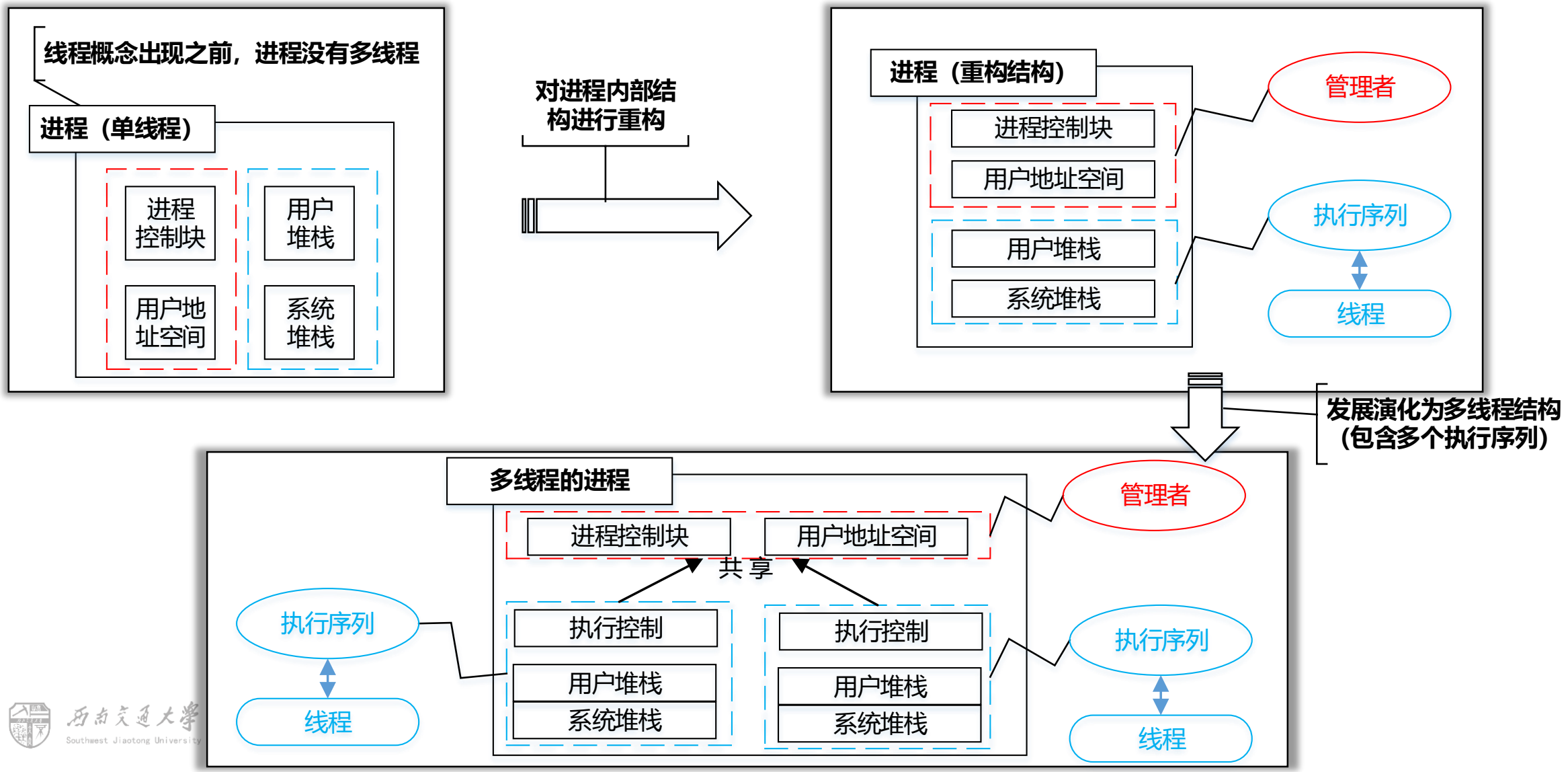


- **Concurrency :** Processes can execute concurrently, and the concurrency performance of processes improves resource utilization and system efficiency (multiple processes take turns occupying the processor).
- **Independence:** A process is the basic unit for resource allocation and protection in a system - different processes do not share memory addresses.
- **Structurality:** A process consists of a data set and the programs running on it, including: program blocks, data blocks and process control blocks.

Processes and Threads

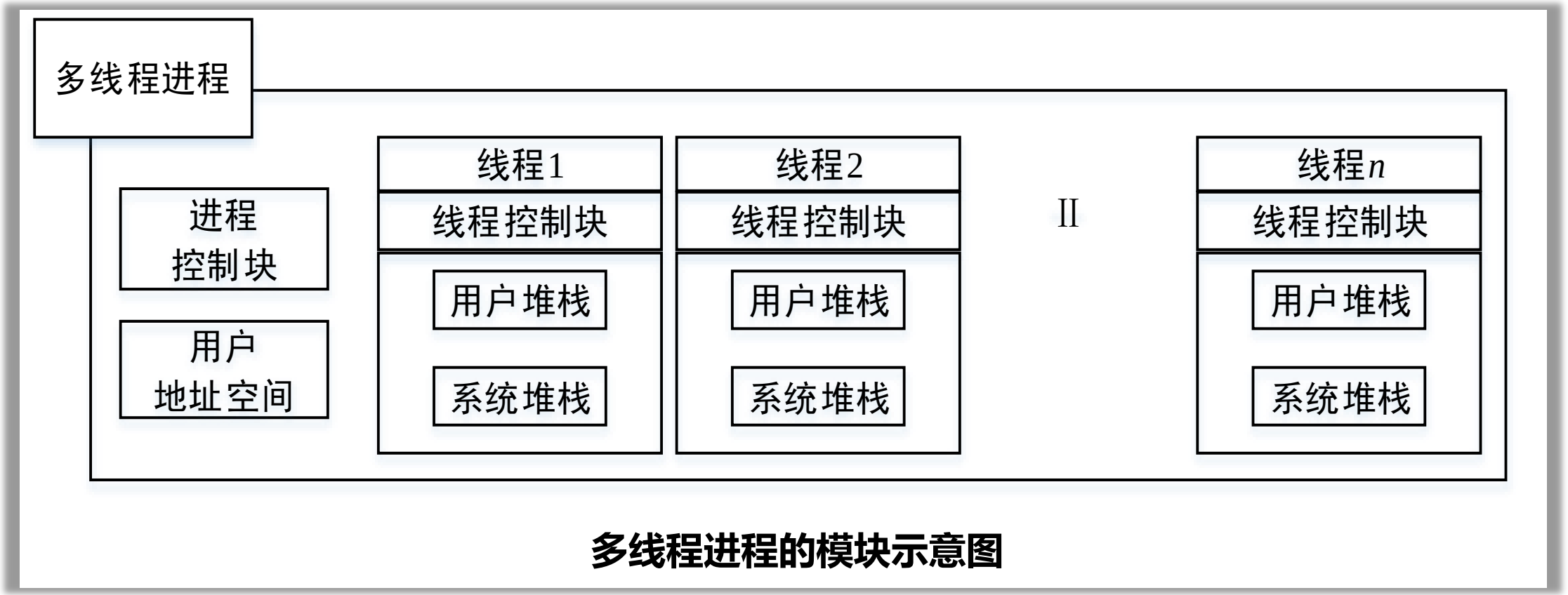
- Processes have two characteristics:
  - Resource Ownership
    - ✓ Process includes a virtual address space to hold the process image
    - ✓ OS performs a protection function to prevent unwanted interference between processes with respect to resources
    - ✓ Protected access to processors, other processes, files and I/O resources
  - Scheduling/Execution
    - ✓ Follows an execution path that may be interleaved with other processes
    - ✓ A process has an execution state (Running, Ready, etc.) and a dispatching priority
    - ✓ A process is scheduled and dispatched by the OS

Evolution of Process



西南交通大学 Southwest Jiaotong University

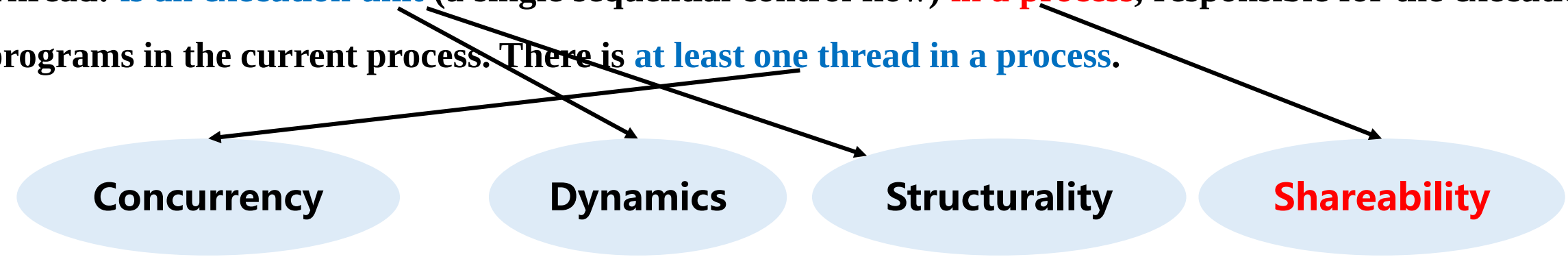
Diagram of Process with Multi-Threads



多线程进程的模块示意图

Process Properties/Attributes

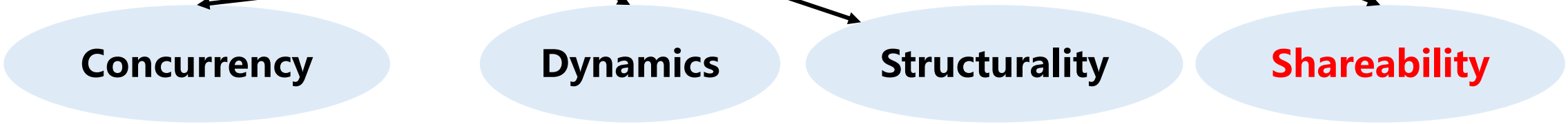
Thread: is an execution unit (a single sequential control flow) in a process, responsible for the execution of programs in the current process. There is at least one thread in a process.



- **Concurrency:** Multiple threads of the same process can execute concurrently or in parallel on one/multiple processors, while concurrent execution between processes evolves into concurrent execution between threads of different processes.
- **Shareability:** All threads within the same process share but do not own the process's state and resources, and reside in the same main memory address space of the process, allowing access to the same data - multiple threads within the same process share memory addresses (such as global variables and file access permissions).

Process Properties/Attributes

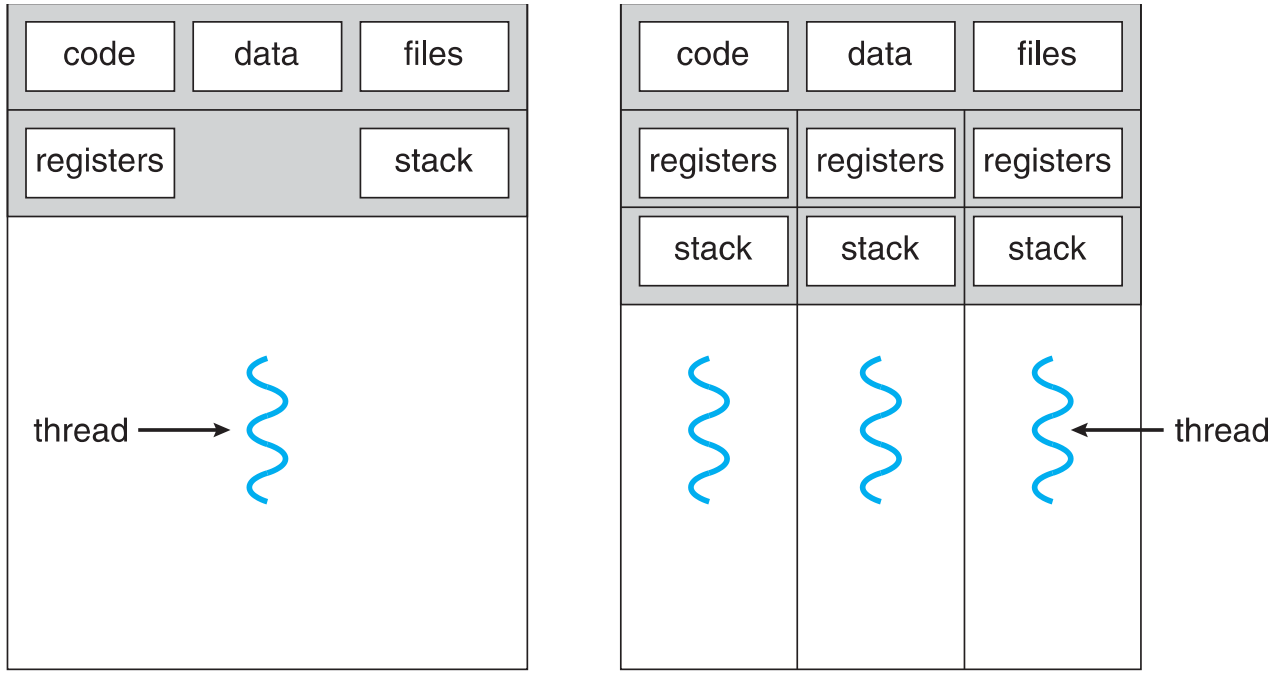
Thread: is an execution unit (a single sequential control flow) in a process, responsible for the execution of programs in the current process. There is at least one thread in a process.



- **Dynamics:** A thread is an execution process of a program on a corresponding dataset, with its own lifecycle. When each process is created, at least one thread is created for it at the same time, and other threads can be created if necessary.
- **Structurality:** Threads are the basic scheduling and dispatching units in the operating system, with unique identifiers and thread control blocks

Processes and Threads

- The unit of dispatching is referred to as a thread or lightweight process
- The unit of resource ownership is referred to as a process or task
- Multithreading - The ability of an OS to support multiple, concurrent paths of execution (多个并发执行路径) within a single process



single-threaded process　　multithreaded process

西南交通大學 Southwest Jiaotong University

Single Threaded Approaches

- A single thread of execution per process
  - the concept of a thread is not recognized
- MS-DOS
- Traditional Unix

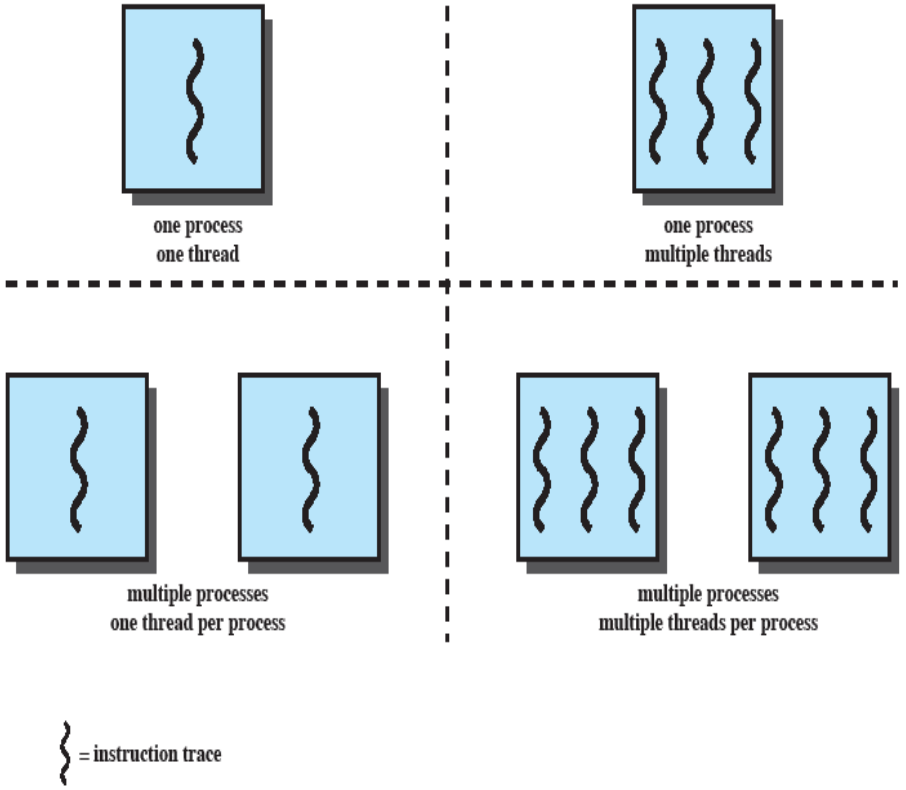


Figure 4.1 Threads and Processes [ANDE97]

Multithreaded Approaches

- The right half of Figure 4.1 depicts multithreaded approaches
- Java is an example of a system of one process with multiple threads
- Windows, Solaris and modern Unix are multiple processes with multiple threads

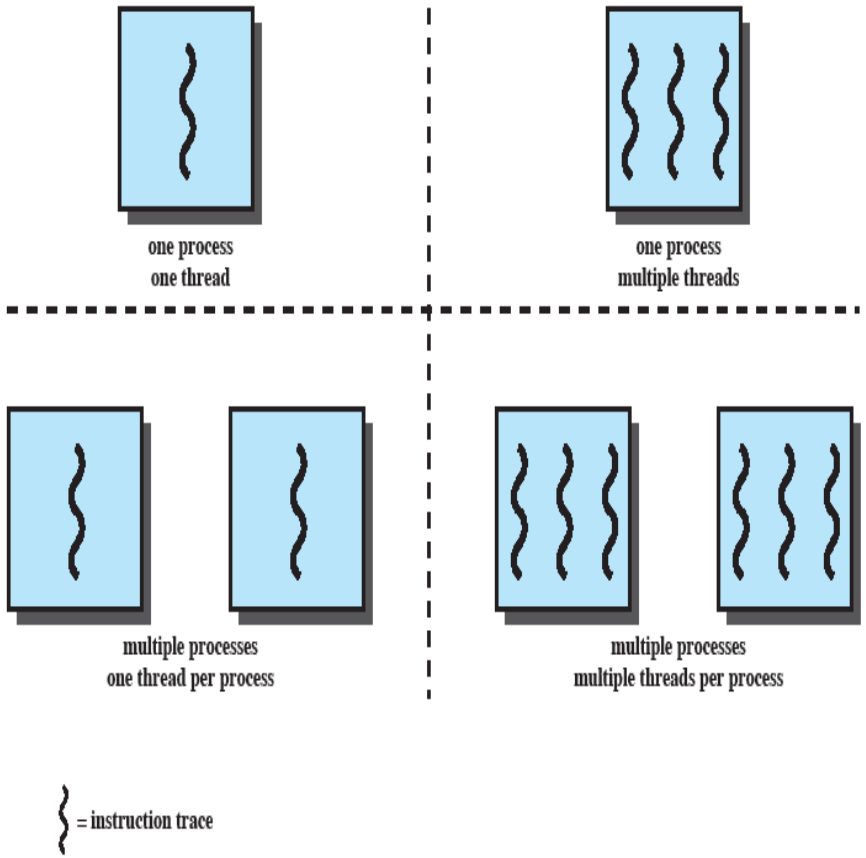
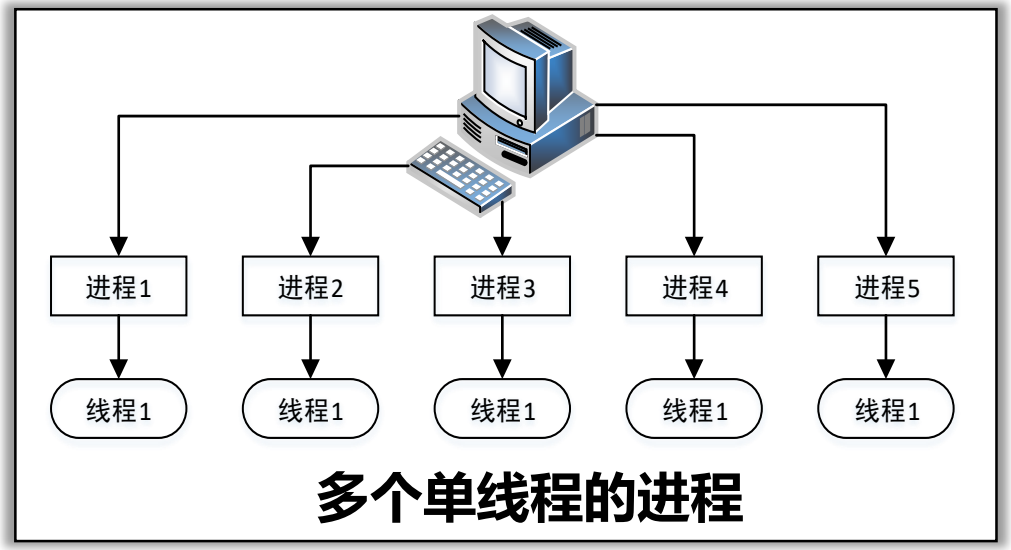
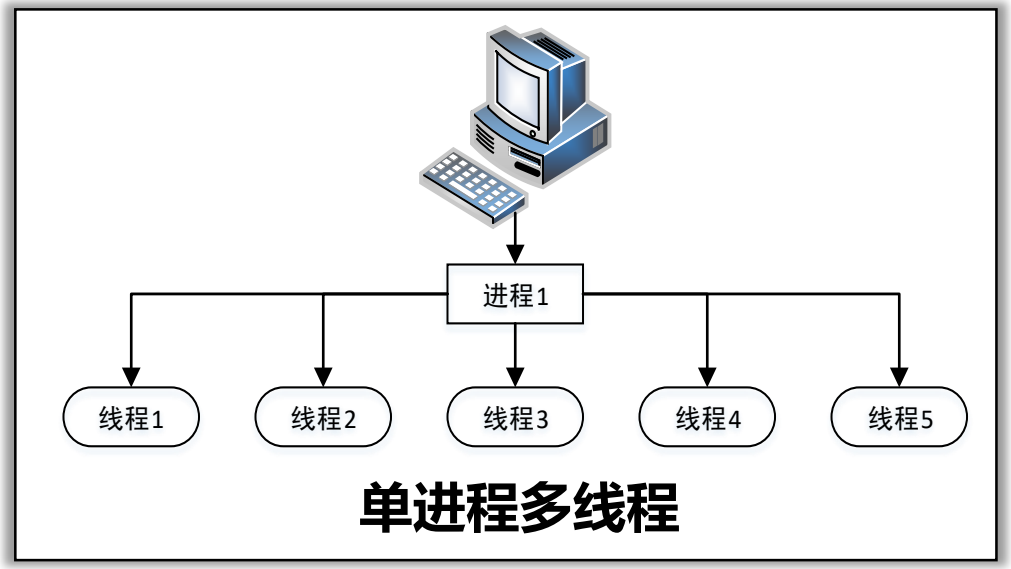
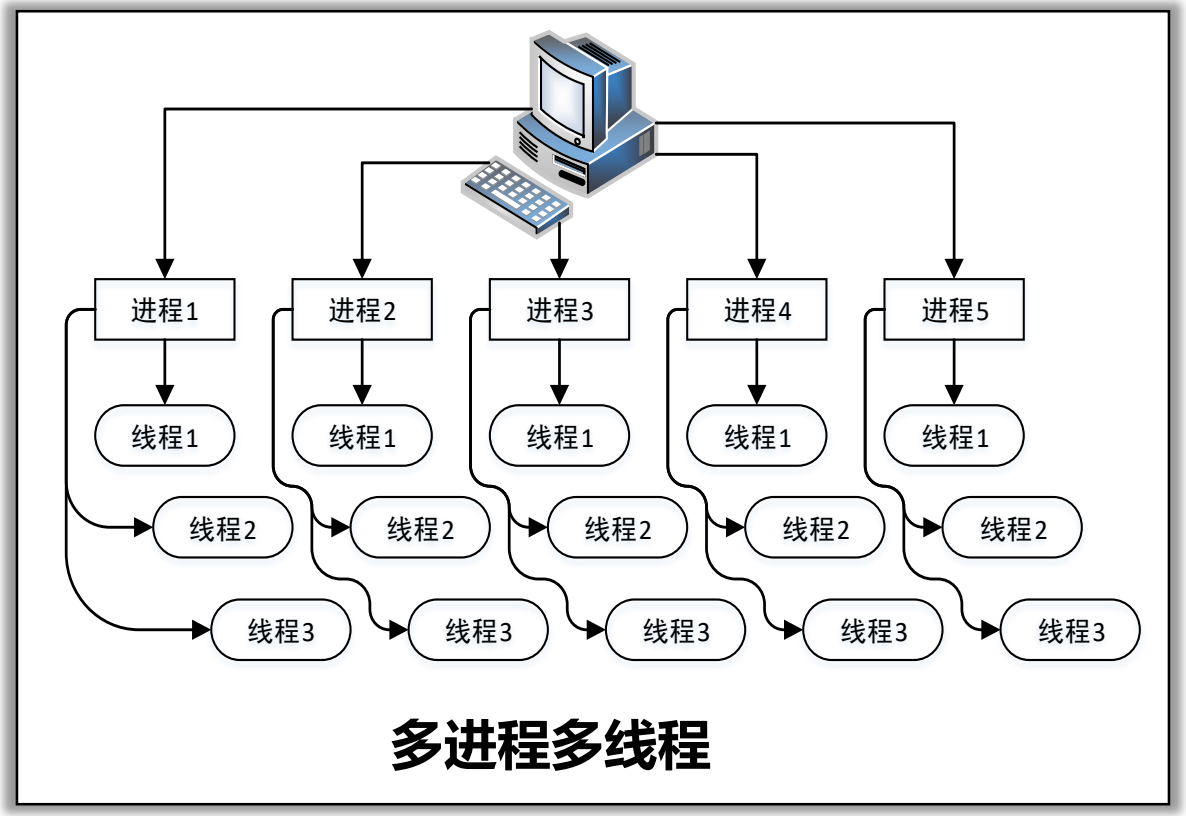


Figure 4.1 Threads and Processes [ANDE97]

Structure of Process and Thread







只确定了进程线程数量，未限定时空分布

西南交通大学
Southwest Jiaotong University

One or More Threads in a Process

- Each thread has:
  - an execution state (Running, Ready, etc.)
  - saved thread context when not running
  - an execution stack
  - some per-thread static storage for local variables
  - access to the memory and resources of its process (all threads of a process share this)

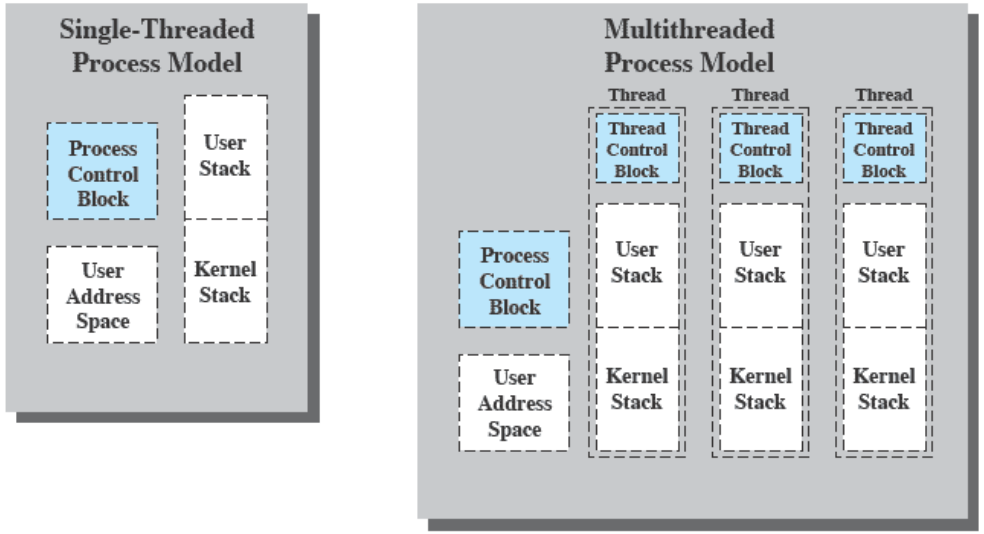


Figure 4.2 Single Threaded and Multithreaded Process Models

Benefits of Threads

- Less time to create a new thread than a process (and less overhead)
- Less time to terminate a thread than a process (and overhead)
- Switching between two threads takes less time than switching between processes
- Threads enhance efficiency in communication between programs by avoiding kernel intervention: In order to achieve collaboration, processes or threads need to exchange data. For threads that automatically share the same address space, all global data can be freely accessed without any special facilities to achieve data sharing. Process communication is quite complex, requiring the use of facilities such as communication mechanisms, message buffering, and pipeline mechanisms, as well as calling kernel functions to achieve it. At the same time, the efficiency of thread communication is also very high.



Multi threads share address within the process

Thread Usage

- Multiprocessor system
  - Multiple threads in the same process can be executing simultaneously on different processors.
- Uniprocessor system
  - To simplify the structure of a program that is logically doing several different functions.
    - ✓ Foreground and background work
    - ✓ Asynchronous processing （异步处理）
    - ✓ Speed of execution
    - ✓ Modular program structure

Thread Execution States

- The key states for a thread are:
  - Running
  - Ready
  - Blocked
- Thread operations associated with a change in thread state are:
  - Spawn （产卵、派生）
  - Block
  - Unblock
  - Finish

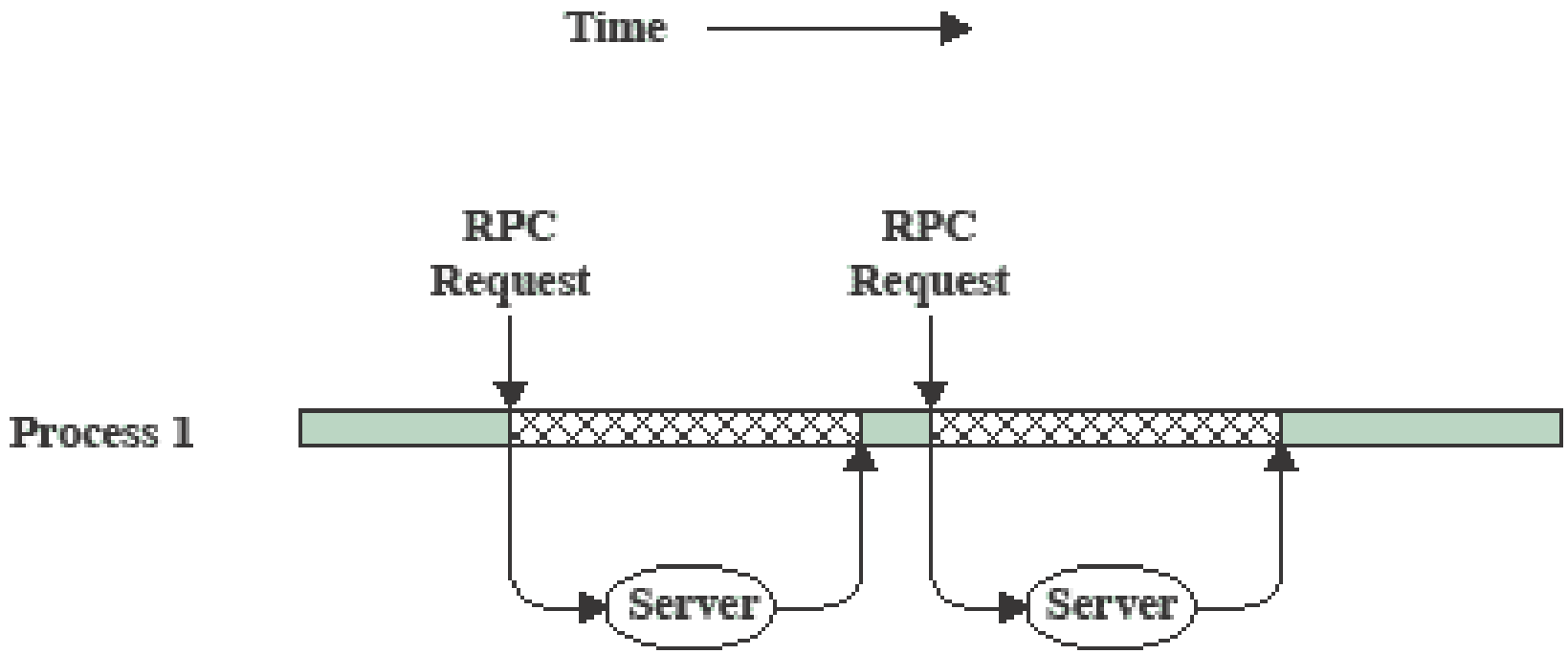


Why not suspend state? How about swapping a thread?

Thread Execution

- In an OS that supports threads
  - scheduling and dispatching is done on a thread basis
- Most of the state information dealing with execution
  - States are maintained in thread-level data structures
  - suspending a process involves suspending all threads of the process
  - termination of a process terminates all threads within the process
- It is necessary to synchronize(同步) the activities of the various threads
  - all threads of a process share the same address space and other resources
  - any alteration of a resource by one thread affects the other threads in the same process

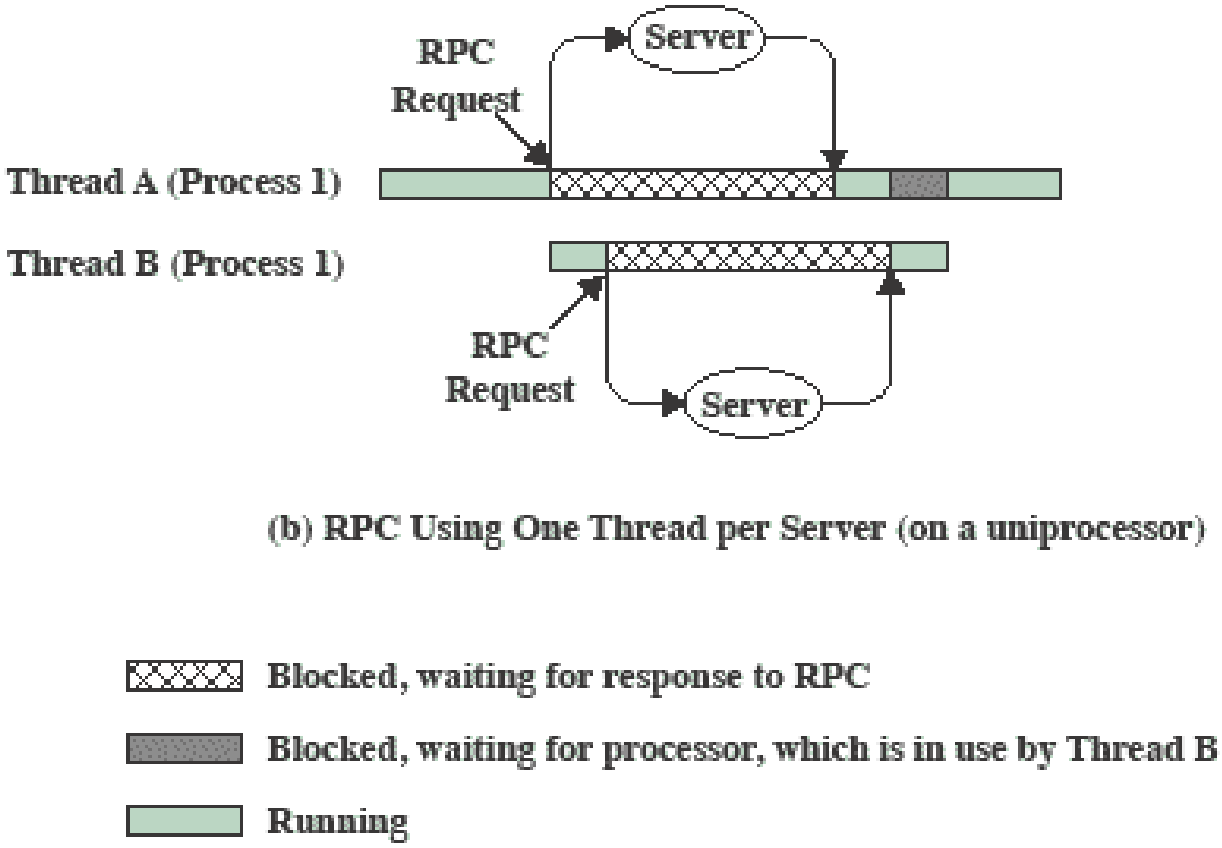
RPC Using Single Thread



(a) RPC Using Single Thread

RPC：Remote Procedure Call

RPC Using One Thread per Server



(b) RPC Using One Thread per Server (on a uniprocessor)

Multithreading on a Uniprocessor

?

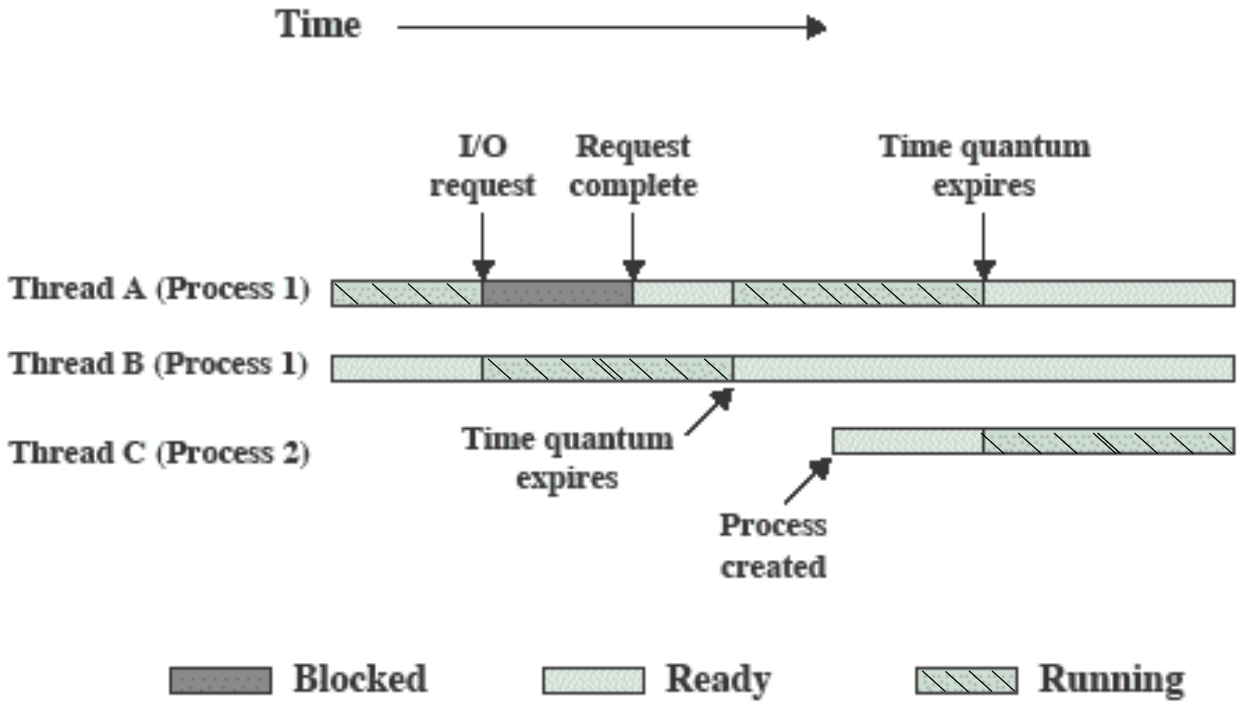
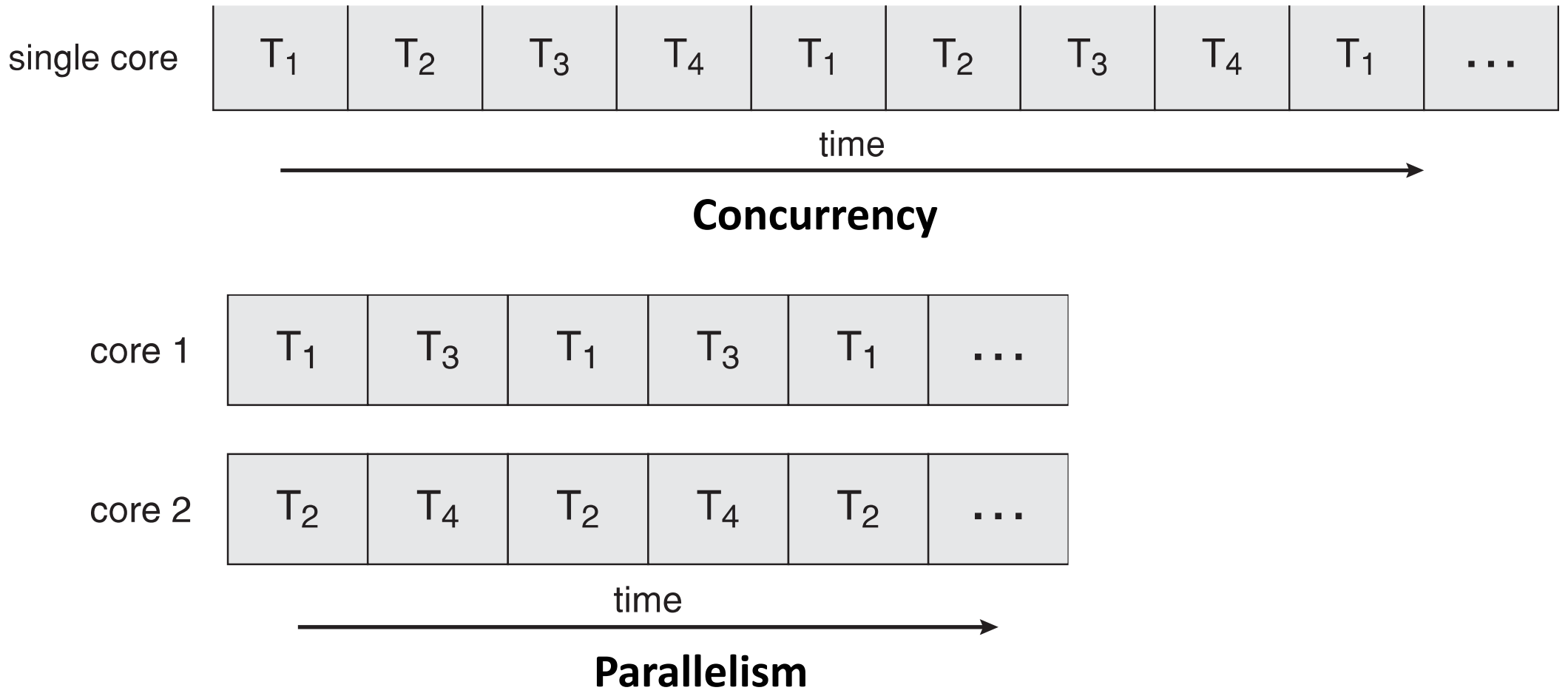
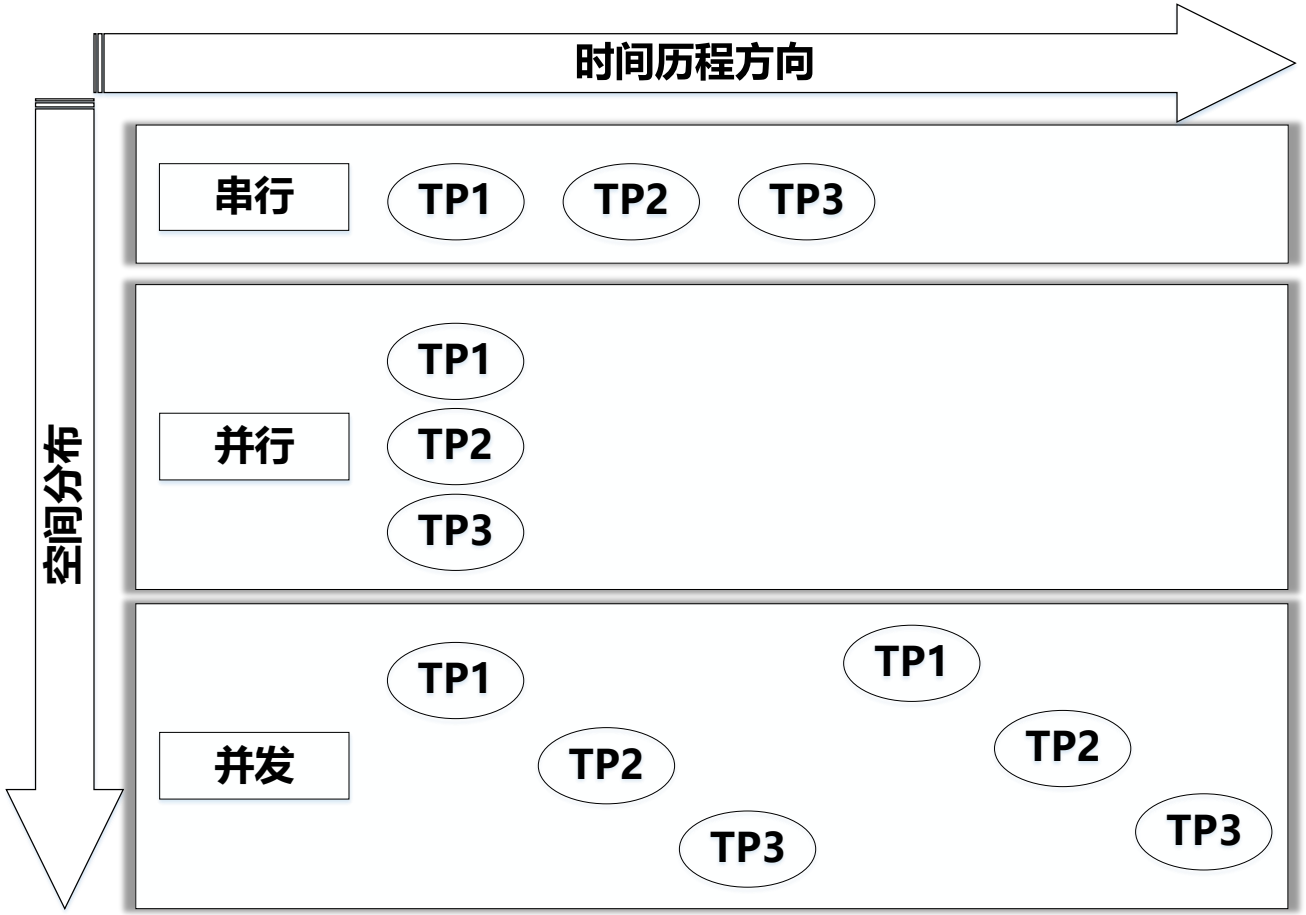


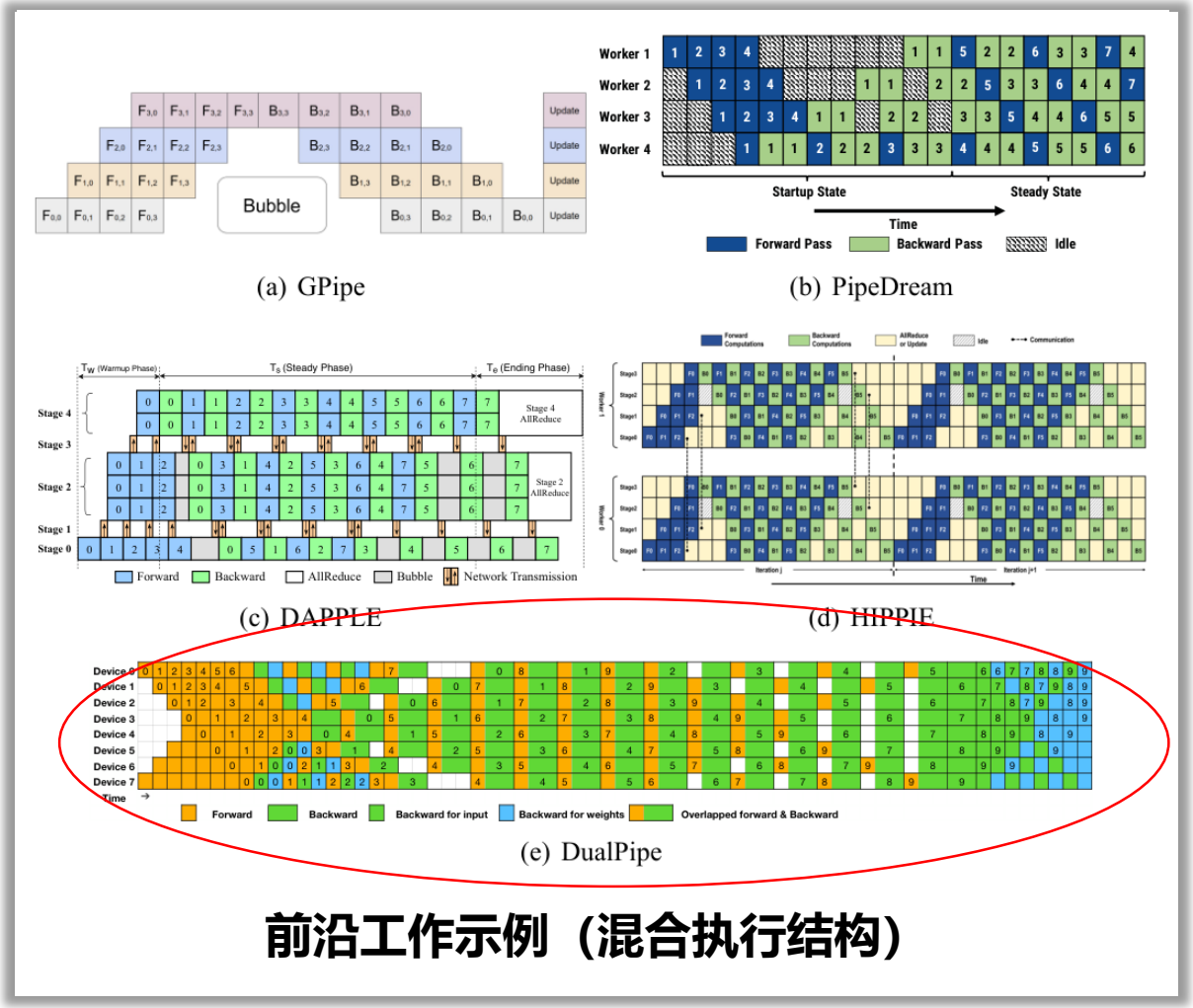
Figure 4.4 Multithreading Example on a Uniprocessor

Concurrency（并发）v.s. Parallelism（并行）



Concurrency（并发） v.s. Parallelism（并行）





前沿工作示例（混合执行结构）

3.2 Types of Threads

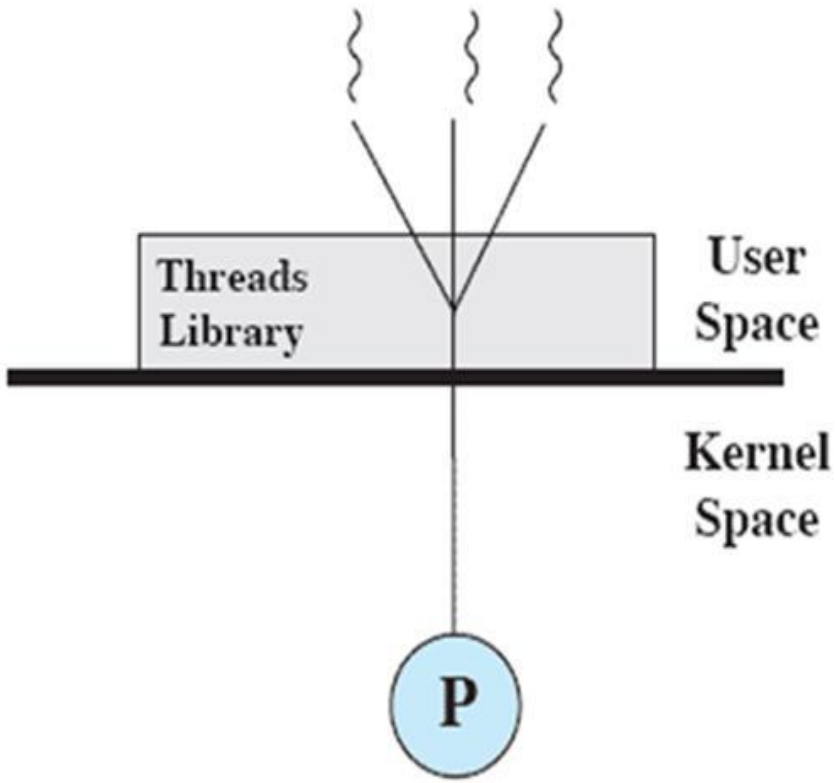
- User Level Thread (ULT)
- Kernel level Thread (KLT)



User mode v.s kernel mode
User space v.s kernel space

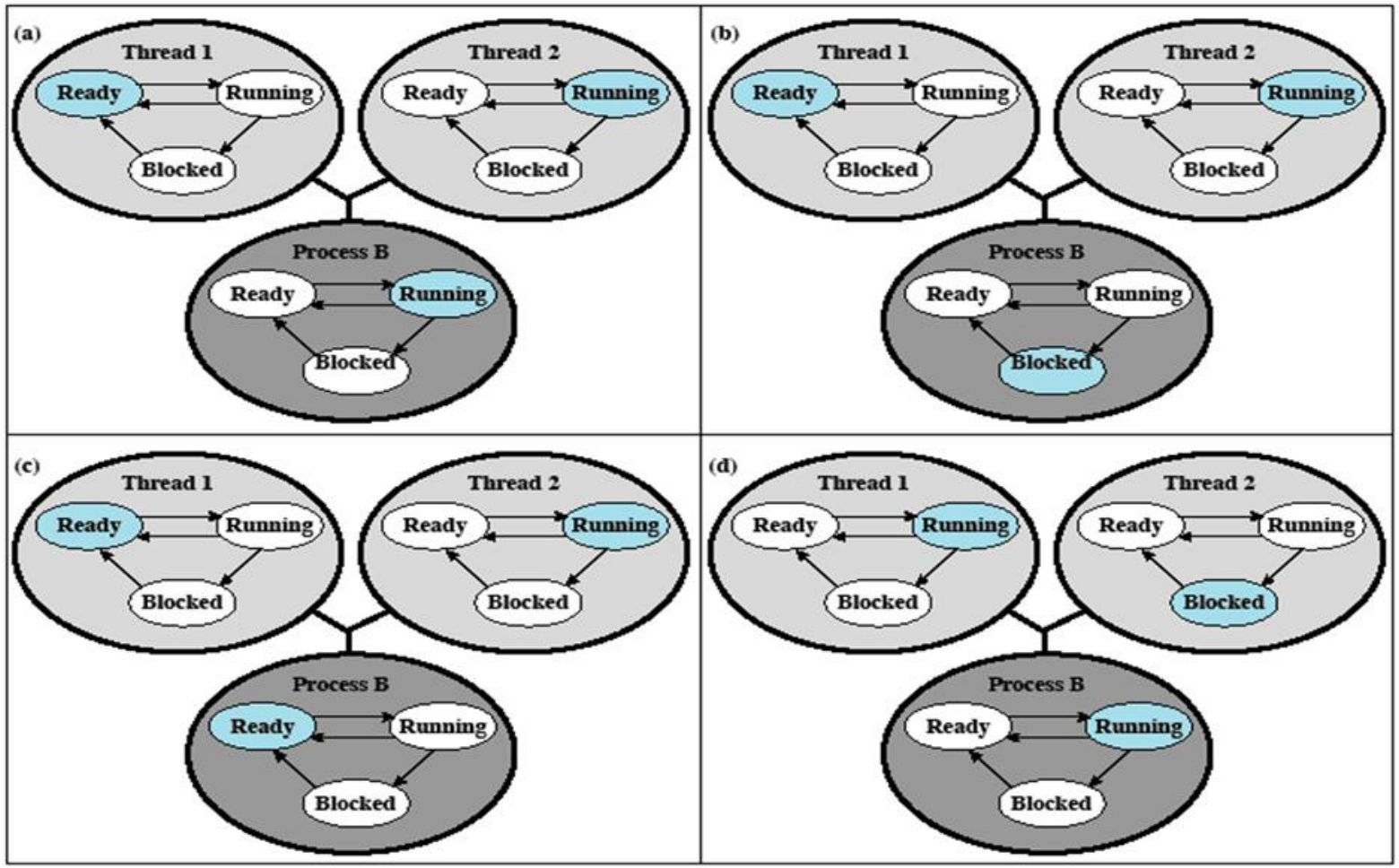
User-Level Threads (ULTs)

- All thread management is done by the application
- The kernel is not aware of the existence of threads
- Thread library (线程库) provides programmer with API for creating and managing threads
  - POSIX Pthreads
  - Win32 threads
  - Java threads



(a) Pure user-level

Relationships Between ULT States and Process States



Colored state
is current state

Figure 4.7 Examples of the Relationships Between User-Level Thread States and Process States



Advantages of ULTs

- Thread switching does not require kernel mode privileges
- Scheduling can be application specific
- ULTs can run on any OS

Disadvantages of ULTs

- when a ULT executes a system call, not only is that thread blocked, but all of the threads within the process are blocked

- In a pure ULT strategy, a multithreaded application cannot take advantage of multiprocessing



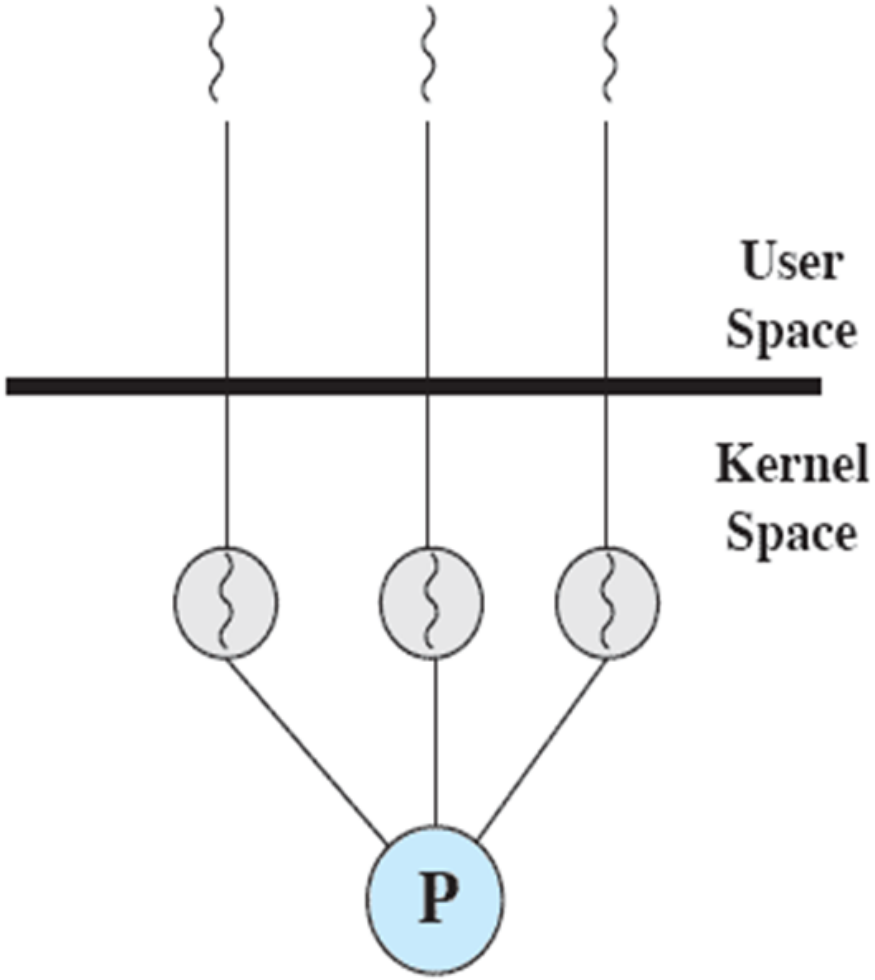
For computationally intensive applications, multithreading is meaningless

Overcoming ULT Disadvantages

- Jacketing
  - converts a blocking system call into a non-blocking system call (by additionally checking the status of resources)
- Writing an application as multiple processes rather than multiple threads

Kernel-Level Threads (KLTs)

- Thread management is done by the kernel
  - no thread management is done by the application
  - Windows is an example of this approach
- Examples – virtually all general purpose OS
  - Windows
  - Solaris
  - Linux
  - Tru64 UNIX
  - Mac OS X



(b) Pure kernel-level

Advantages of KLTs

- The kernel can simultaneously schedule multiple threads from the same process on multiple processors
- If one thread in a process is blocked, the kernel can schedule another thread of the same process
- Kernel routines (内核例程) can be multithreaded

Disadvantage of KLTs

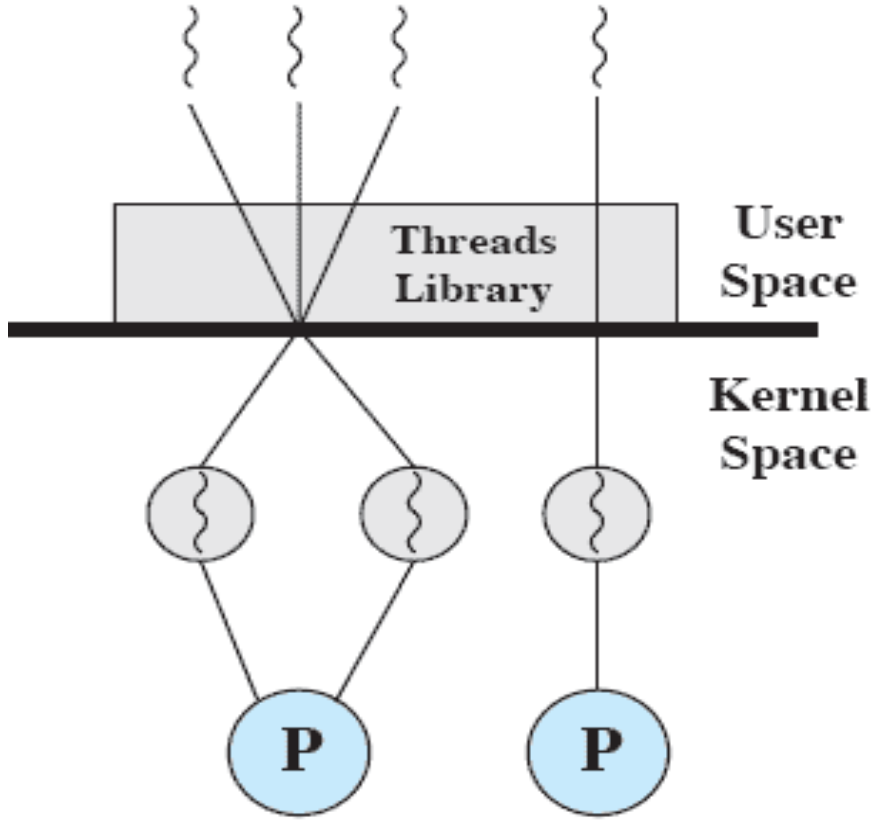
- The transfer of control from one thread to another within the same process requires a mode switch to the kernel

Operation	User-Level Threads	Kernel-Level Threads	Processes
Null Fork	34	948	11,300
Signal Wait	37	441	1,840

Combined Approaches

- Thread creation is done in the user space
- Bulk of scheduling and synchronization of threads is by the application

Threads:Processes	Description	Example Systems
1:1	Each thread of execution is a unique process with its own address space and resources.	Traditional UNIX implementations
M:1	A process defines an address space and dynamic resource ownership. Multiple threads may be created and executed within that process.	Windows NT, Solaris, Linux, OS/2, OS/390, MACH
1:M	A thread may migrate from one process environment to another. This allows a thread to be easily moved among distinct systems.	Ra (Clouds), Emerald
M:N	Combines attributes of M:1 and 1:M cases.	TRIX



(c) Combined

3.3 Multicore / Multithread

- Multithreading
- Thread functionality

Database Workloads on Multiple-Processor Hardware

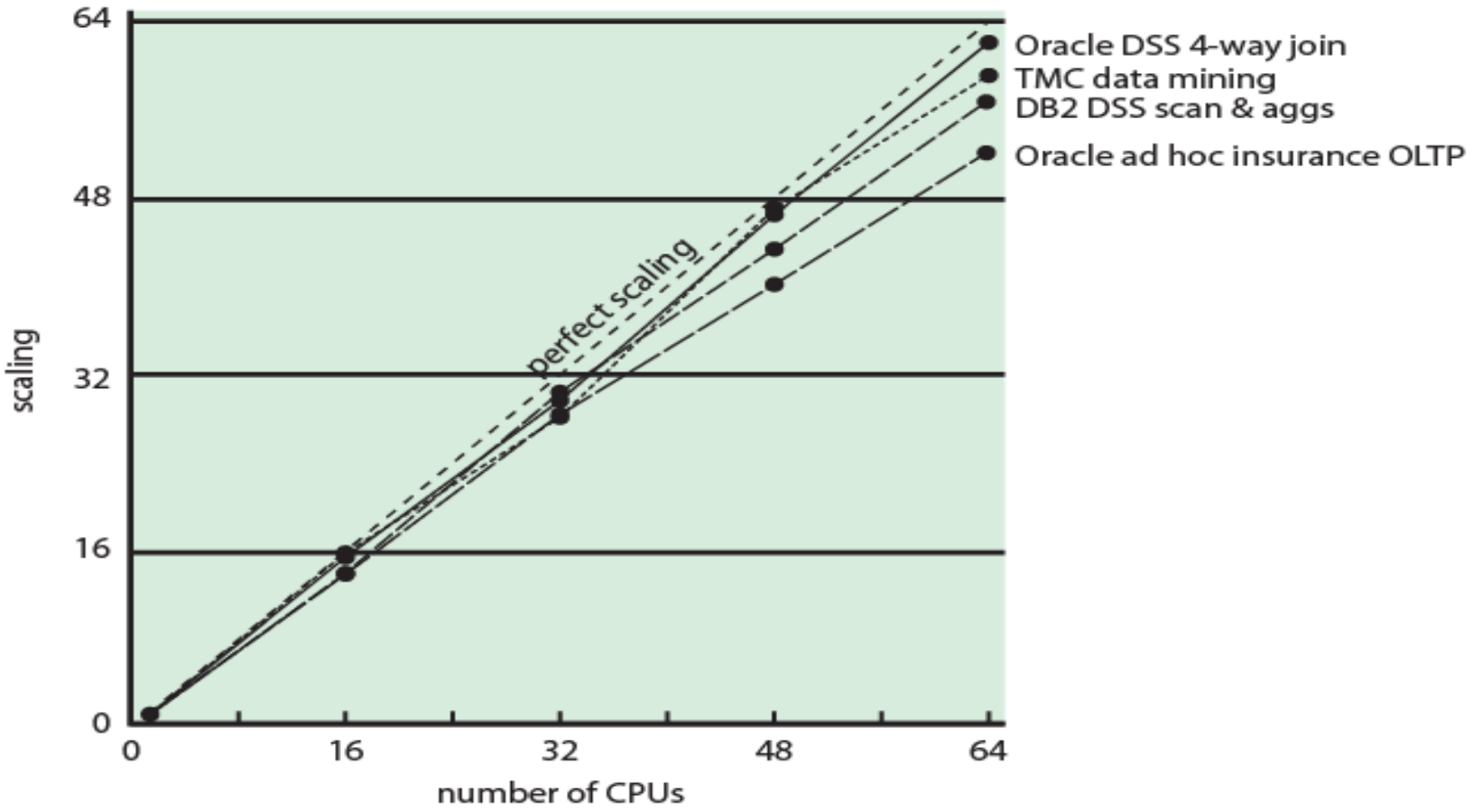


Figure 4.8 Scaling of Database Workloads on Multiple Processor Hardware

西南交通大學
Southwest Jiaotong University

Amdahl's Law (阿姆达尔定律)

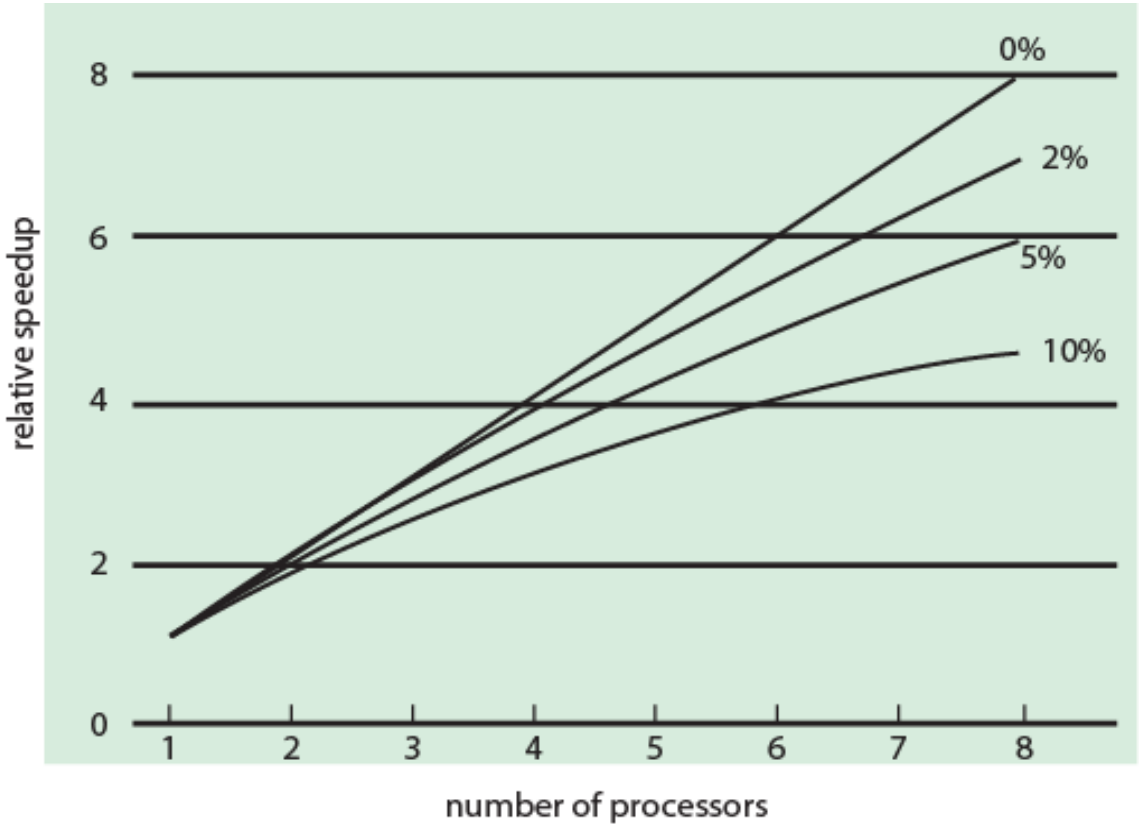
- Identifies performance gains from adding additional cores to an application that has both serial and parallel components

$$speedup \leq \frac{1}{S + \frac{(1-S)}{N}}$$

- *S* is serial portion, *N* processing cores
- As *N* approaches infinity, speedup approaches 1 / *S*

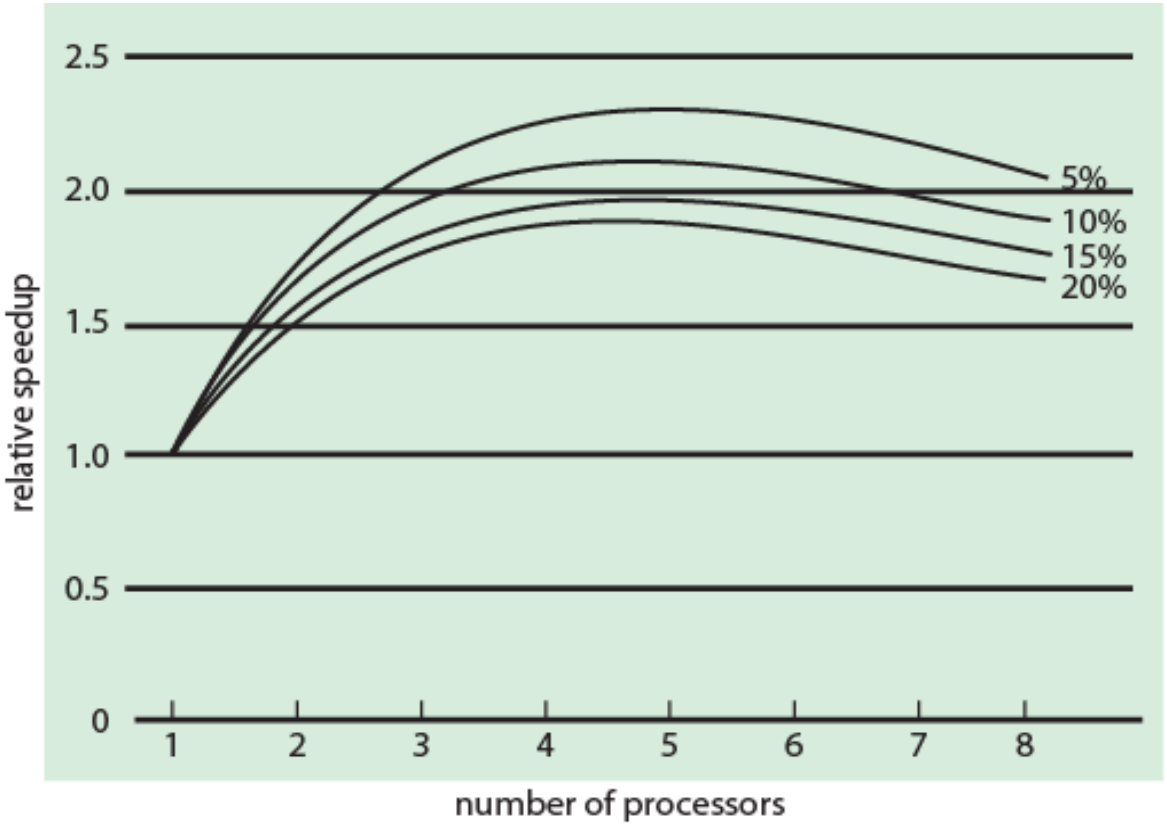
- Serial portion of an application has disproportionate (不成比例) effect on performance gained by adding additional cores

Performance Effect of Multiple Cores



(a) Speedup with 0%, 2%, 5%, and 10% sequential portions

Figure 4.7 (a)



(b) Speedup with overheads

Figure 4.7 (b)

西南交通大學
Southwest Jiaotong University

Benefited Applications from Multicore

- Multithreaded native applications
  - characterized by having a small number of highly threaded processes
- Multiprocess applications
  - characterized by the presence of many single-threaded processes
- Java applications
- Multiinstance applications
  - multiple instances of the application in parallel

Linux Process/Thread

- Traditional Linux supports user-level library functions, eg pthread （POSIX线程, Portable Operating System Interface Threads）
- Modern Linux provides a unique solution that does not recognize a distinction between threads and processes (不区分)
  - User-level threads are mapped into kernel-level processes
  - A new process is created by copying the attributes of the current process
  - The new process can be *cloned* so that it shares resources
  - The clone() call creates separate stack spaces for each process
  - The traditional fork() system call is implemented by Linux as a clone() system call.

Linux Process/Thread Model

- Running
- Ready
- Interruptible(可中断)
- Uninterruptible
- Stopped(停止)
- Zombie (僵死)

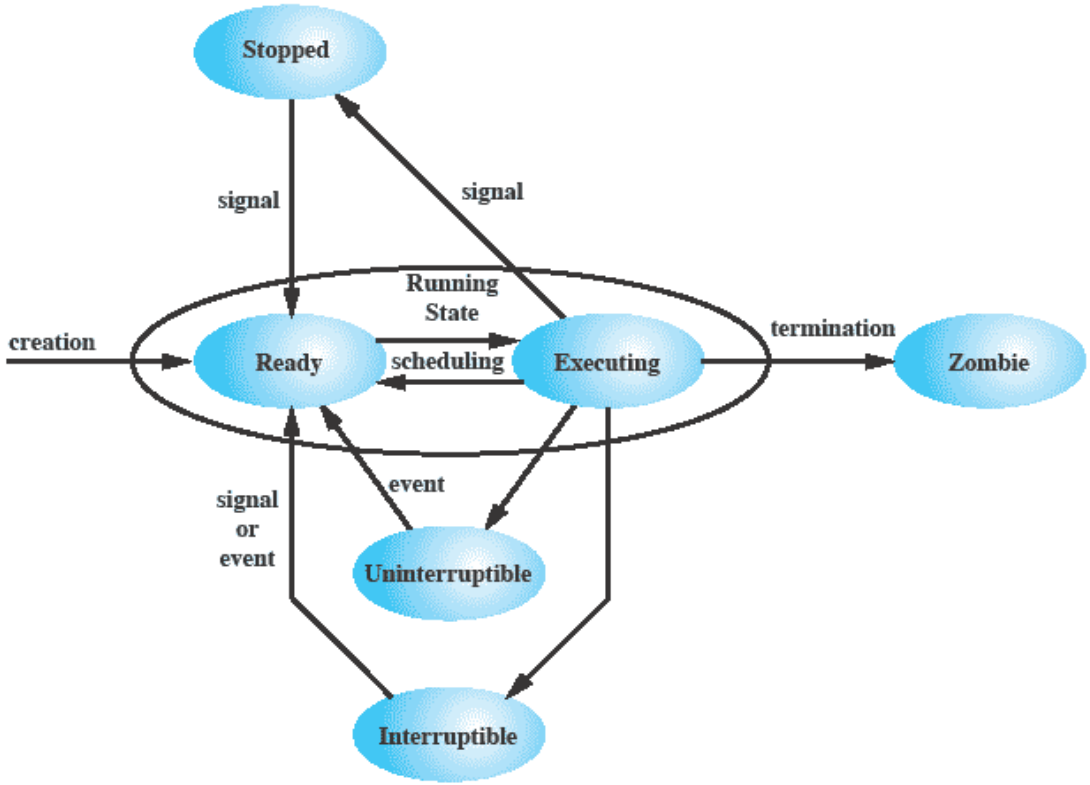


Figure 4.18 Linux Process/Thread Model

Pthread Library

- POSIX (IEEE 1003.1c) specification of threads
  - API supported by most UNIX systems
  - Can be implemented in user or kernel space
  - Threads have properties including: Identifier, stack size, scheduling parameters, etc.

- Primary functions
  - pthread_create
  - pthread_exit
  - pthread_join
  - pthread_yield
  - pthread_attr_init (Create and initialise thread attribute structure)
  - pthread_attr_destroy (Remove thread attribute structure)

POSIX Pthreads

线程调用	描述
Pthread_create	创建一个线程
Pthread_exit	结束调用的线程
Pthread_join	等待一个特定的线程退出
Pthread_yield	释放CPU来运行另外一个线程
Pthread_attr_init	创建并初始化一个线程的属性结构
Pthread_attr_destroy	删除一个线程的属性结构

Some function calls of pthread

An Example of Pthread

```c
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#define NUMBER_OF_THREADS 10

void *print_hello_world(void *tid)
{ /* Print thread's identifier */
    printf("Greetings from thread %d0", tid);
    pthread_exit(NULL);
}

int main (int argc, char* argv[])
{ /* create threads and exit */
    pthread_t threads[NUMBER_OF_THREADS];
    int status, i;

    for(i=0; i<NUMBER_OF_THREADS; i++) {
        printf("Main: Creating thread %d0", i);
        status = pthread_create(&threads[i], NULL, print_hello_world, (void*)i);

        if(status != 0) {
            printf("Error from pthread: %d0", status);
            exit(-1);
        }
    }
    exit(NULL);
}
```



Summary

- User-level threads
  - created and managed by a threads library that runs in the user space of a process
  - a mode switch is not required to switch from one thread to another
  - only a single user-level thread within a process can execute at a time
  - if one thread blocks, the entire process is blocked
- Kernel-level threads
  - threads within a process that are maintained by the kernel
  - a mode switch is required to switch from one thread to another
  - multiple threads within the same process can execute in parallel on a multiprocessor
  - blocking of a thread does not block the entire process
- Process/related to resource ownership
- Thread/related to program execution